

Digital Twin for STM Bus Fleet Monitoring

Carlos Pambo **(2346917)**, Kérian Fiter **(2402537)**, Vanny Katarbarwa **(2349909)**

Presented as requirement for **LOG6953FE**

April 2025



Context

- Public transportation is essential for efficient movement in big cities;
- In Montréal, the Société de Transport de Montréal (STM) manages a large bus fleet for public transportation in the city;
- To improve the operational efficiency of the fleet, and provide a more reliable service, we propose building a digital twin (DT) for the bus fleet of the STM;



Context

The goal of this DT is to:

- I. To **develop** a virtual representation of the STM bus lines (near) real-time telemetry data;
- II. To **develop** a visualization tool for the buses;
- III. To **develop** a GPS telemetry validation service;
- IV. To **develop** an alert service to notify STM teams of potential telemetry issues;
- V. To **recommend** suggestions for future work that follows the work reported in project;



Architecture

We proposed an architecture of the DT that includes:

- **Data/Models:** The DT ingests (near) real-time GPS telemetry (coordinates, bearing, speed) from the STM bus fleet through an API;
- **Enablers:** include RabbitMQ for messaging, MongoDB for storage, and a data ingestion pipeline from STM API. These enables pre-process, and distribute data to all other subscribing services of the DT;
- **Services:** These include a visualization service, a GPS telemetry validation service with deviation computation, and a notification service;
- Each of these services accesses data using a Pub/Sub architecture;

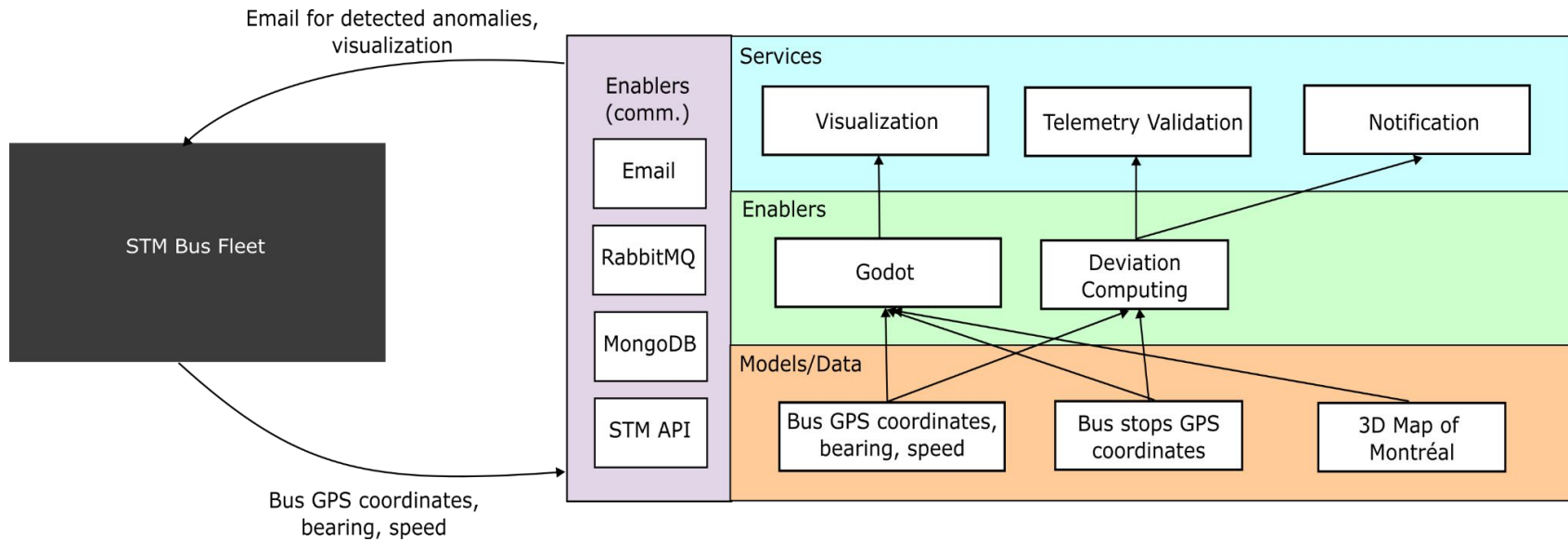


Fig. 1: DT architecture constellation

Services

The following services were built for this digital twin:

- **Telemetry Validation Service:** a *model-free* algorithm to classify GPS telemetry data from the buses;
- **Notification Service:** The service uses SMTP to send emails to a pre-configured list of recipients, when GPS telemetry anomalies are detected;
- **Route Visualization:** this is 3D visualization of the buses in the city, based on the GPS telemetry data;

Algorithm 3.1: GPS telemetry validation algorithm

Input : Bus line **B**, List of GPS coordinates (**C**)

Output: List of validated GPS coordinates (**C'**)

```

1 Initialize C' with C;
2 for  $i = 1$  to  $C_{size}$  do
3     Select coordinates in Ci and save it in X;
4     Ensure all required fields exist in X ;
5     Query the database to get bus stop information for the bus line B and save it in R ;
6     if Bus stops in R are found then
7         Determine the coordinates status by comparing positions ;
8         Extract bus stops from R, or use an empty list if not present and store it in T ;
9         Get the maximum allowed distance from T and store it in M ;
10        if T is not empty then
11            Sort the routes in T ;
12            Get the bus current stop sequence from Ci and store it in B ;
13            Calculate the distance between the bus stop and the bus using latitude and
                longitude coordinates and store it in D ;
14            if  $D \leq THRESHOLD$  then
15                Update C' coordinate status at index i to OK ;
16                 $i++$  ;
17            if  $THRESHOLD \leq D \leq MAX\_ALLOWED\_DISTANCE$  then
18                Use the bus stop in T for a new distance calculation and store it in D;
19            if  $D \geq MAX\_ALLOWED\_DISTANCE$  then
20                Reverse the bus stop order in T and recalculate for a distance calculation
                    and store it in D;
21            else
22                if  $D \leq THRESHOLD$  then
23                    Update C' coordinate status at index i to OK ;
24                else
25                    Update C' coordinate status at index i to ANOMALY ;
26                 $i++$  ;
27            else
28                Update C' coordinate status at index i to NOT_FOUND ;
29                 $i++$  ;
30        else
31            Update C' coordinate status at index i to NOT_FOUND ;
32             $i++$  ;
33 return C'

```

Fig. 2: GPS telemetry validation algorithm

Good day Operations Team,

This email outlines the need for immediate corrective actions regarding telemetry data for the following bus lines:

121.

Here is a summary of the issues that required **immediate** actions:

Trip-Id	Line	Timestamp	Bus-Stop	Status	Occupancy
284300929	121	2025-03-29 16:39:39	Station Angrignon	INCOMING_AT	EMPTY
284300945	121	2025-03-29 16:39:38	Station Angrignon	STOPPED_AT	MANY_SEATS_AVAILABLE
284301058	121	2025-03-29 16:39:38	Station Angrignon	INCOMING_AT	EMPTY
284300693	121	2025-03-29 16:39:37	Station Angrignon	INCOMING_AT	EMPTY
284301016	121	2025-03-29 16:39:35	Station Angrignon	STOPPED_AT	MANY_SEATS_AVAILABLE

Please prioritize taking actions to remediate issue(s), if need be confirm the readings of the bus driver.

Regards

Fig. 3: Email notification sample

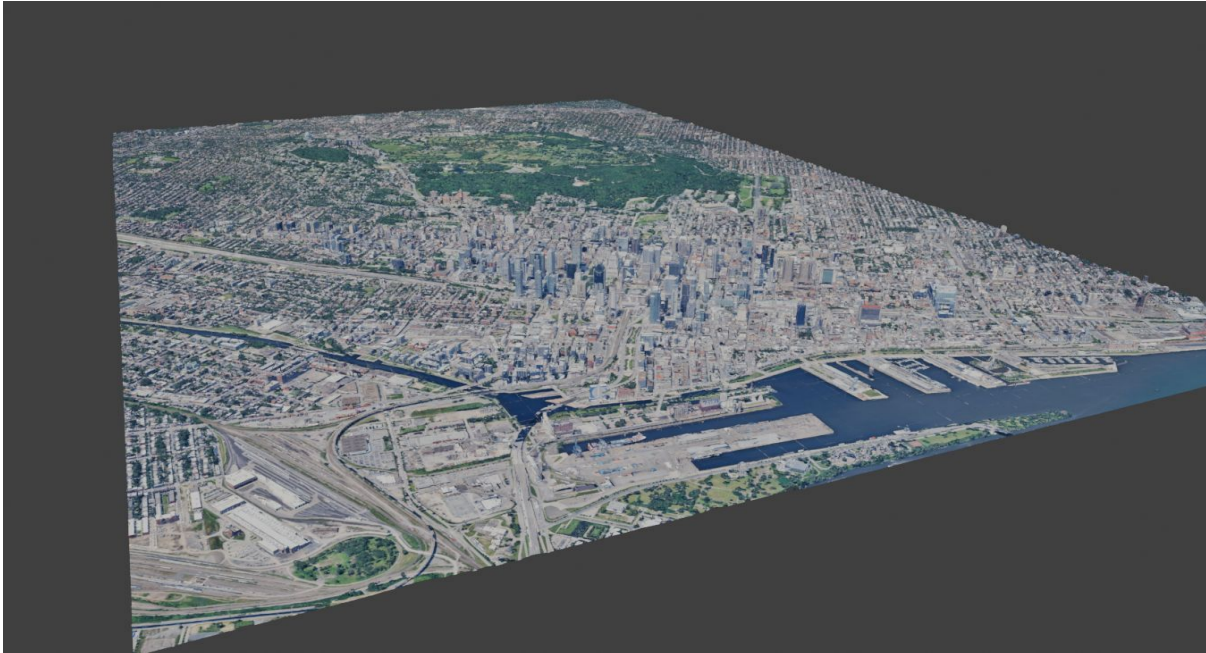


Fig. 3: Montreal model (textured) in Blender

Characteristics:

- Generated in Blender using Google Maps tile API and the Blosm plugin;
- Different levels of details;
- Resource-intensive;
- Heavy textures;

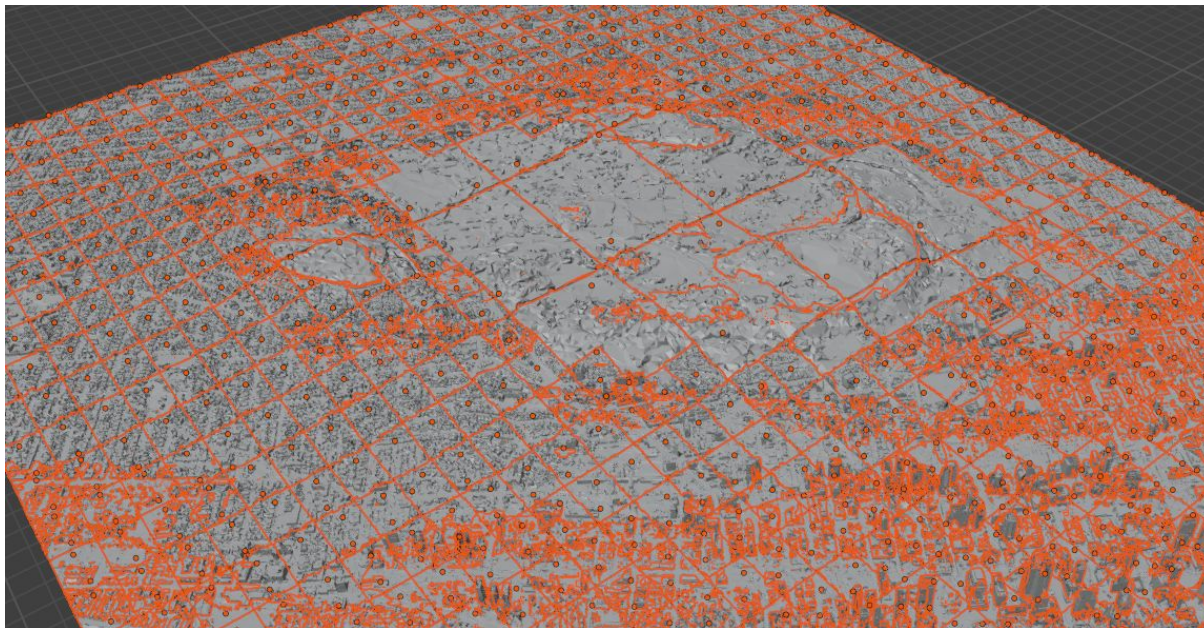
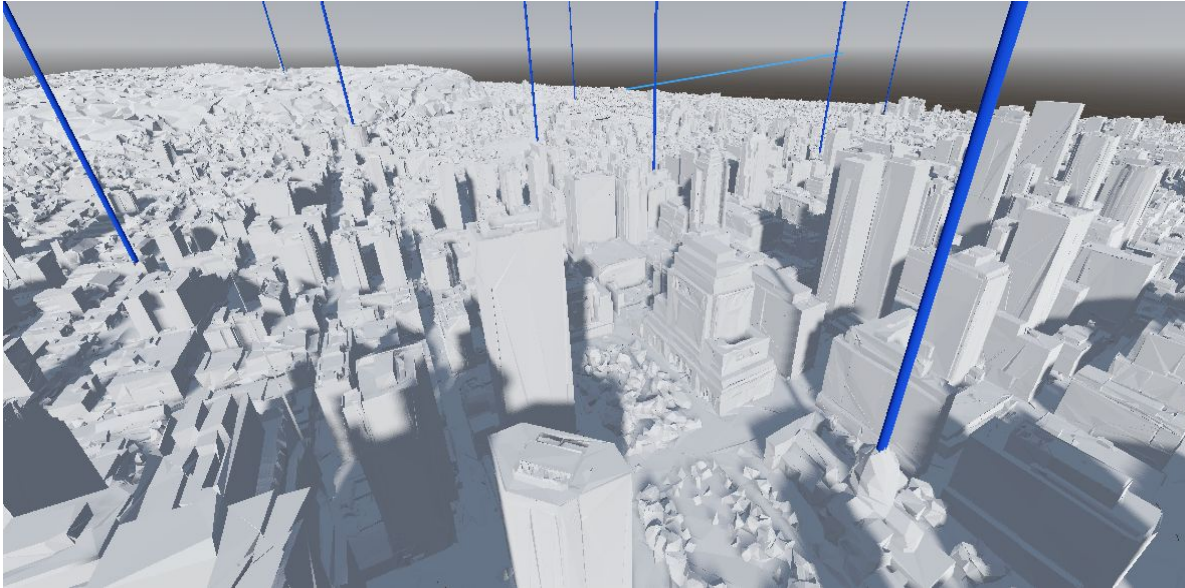


Fig. 4: Map tiles of various sizes in Blender

Characteristics:

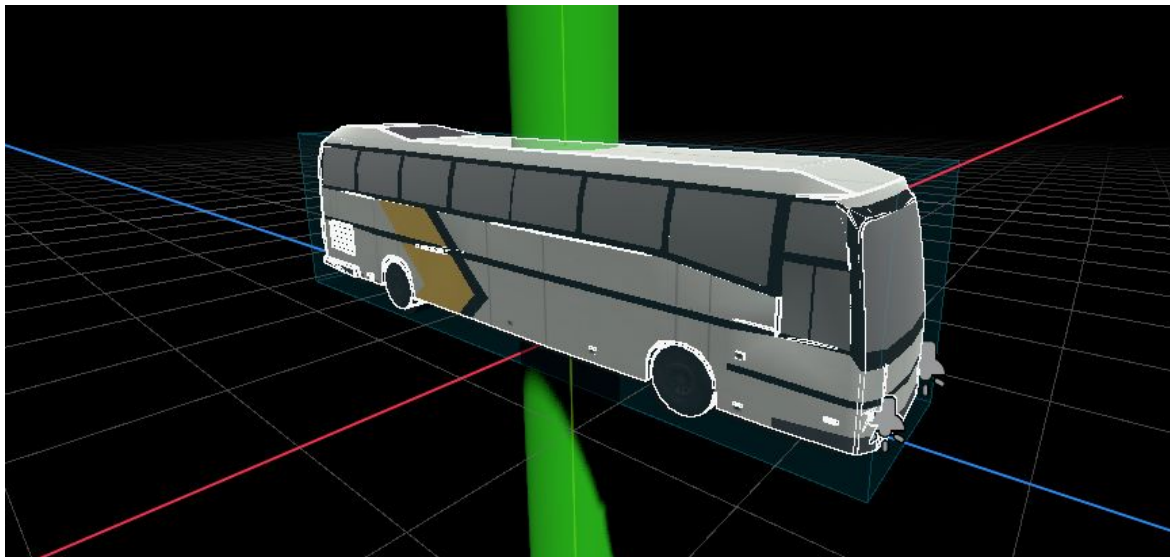
- Chose separated tiles;
- Optimized tiles where there were no roads;
- Removed textures;
- Limited map extent;



Characteristics:

- GPS coordinates - 3D map positions;
- Drift if using one reference point;
- Manually placed reference markers;
- Position triangulation based on three closest markers;

Fig. 5: Reference markers on the map in Godot



Characteristics:

- Bus model;
- Vertical green marker;
- Outline that changes color to signal clickability;
- "Ghost" transparent bus;

Fig. 6: Bus asset in Godot

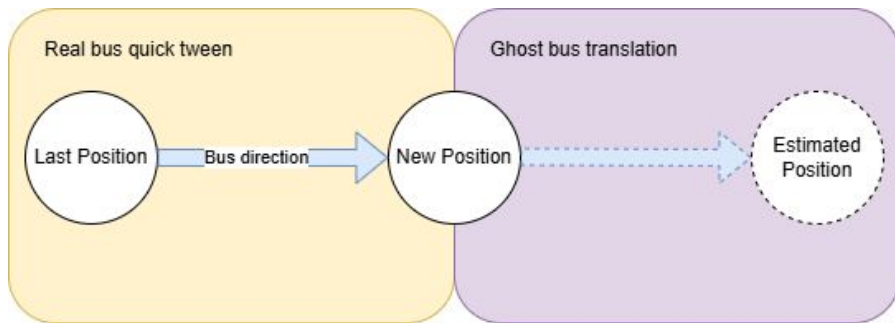


Fig. 7: Bus and ghost bus transition diagram

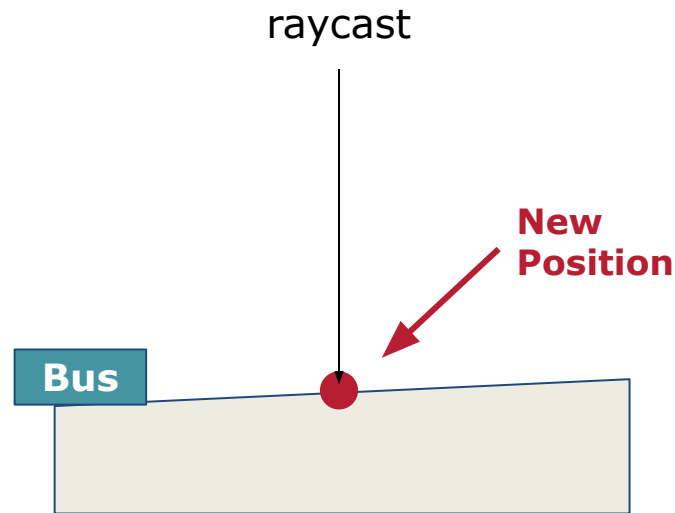


Fig. 8: “Map-sticking” algorithm diagram

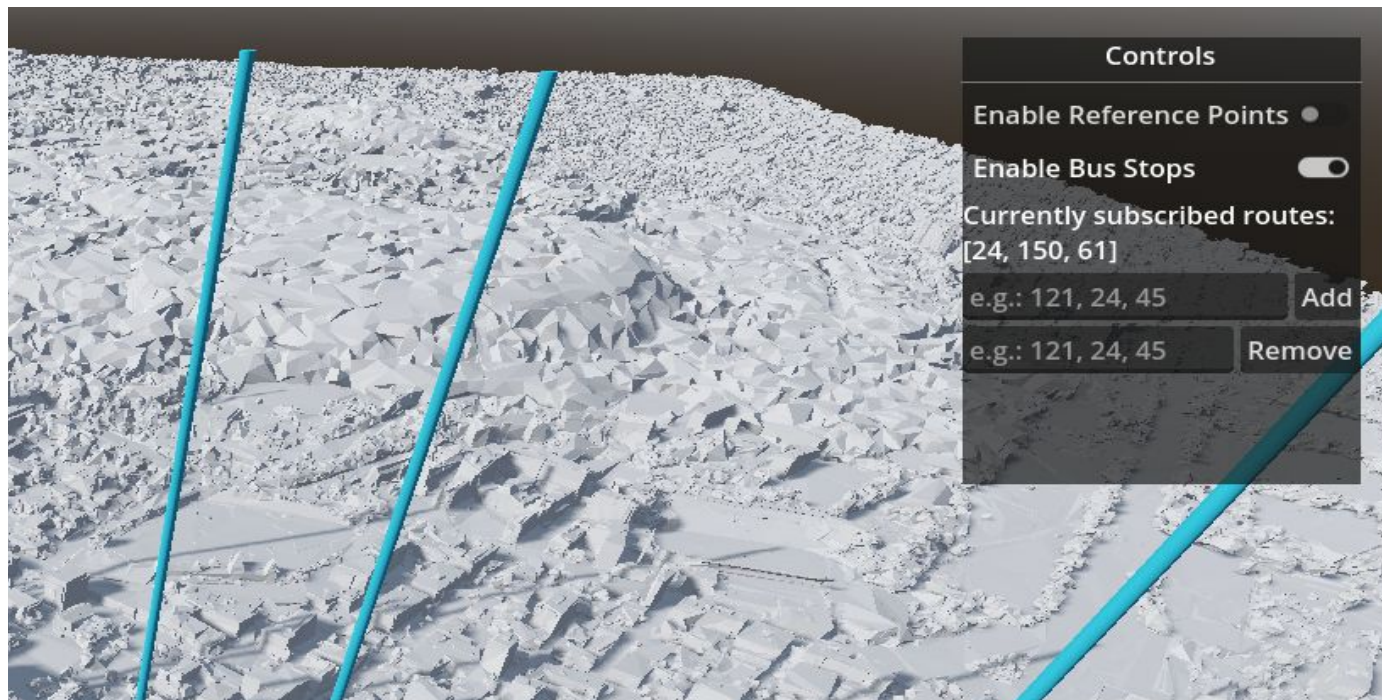


Fig. 9: Visualization controls UI



Takeaways

- STM bus data contained missing entries or inconsistencies, i.e: the API did not provide direction of the bus line;
- STM implementation of rate limiting on its API;
- The team had to develop “*workarounds*” to map coordinates to the 3D map;
- Explore telemetry validation using ML algorithms instead of the current approach;